PERSONAL FINANCE ADVISOR BOT

Professional Project Documentation

Project focus

A web-based personal finance assistant for tracking income and expenses, planning budgets, exploring savings scenarios, and receiving contextual financial guidance.

Prepared as a project blueprint and technical reference

Version 1.0 | October 2026

Document Overview

This document describes the proposed purpose, requirements, architecture, data design, workflows, testing approach, and future scope of the Personal Finance Advisor Bot. It is written as a professional project reference. Feature descriptions should be treated as planned capabilities unless they have been implemented and tested in the actual codebase.

Item	Description
Project name	Personal Finance Advisor Bot
Project type	Web application with AI-assisted financial guidance
Backend	Python with Flask
Data access	SQLAlchemy ORM
Database options	SQLite for local development; PostgreSQL for a deployed or multi-user setup
AI integration	Gemini API or another configured language-model API
Interface	HTML, CSS, Jinja2 templates; charts and voice input can be added where supported
Document status	Architecture and feature blueprint; verify against implementation

Contents

- 1. Executive Summary
- 2. Problem Statement and Objectives
- 3. Scope and Key Features
- 4. Functional and Non-Functional Requirements
- 5. System Architecture
- 6. Technology Stack
- 7. Main User Workflows
- 8. Data Model
- 9. AI Assistant Design and Safety
- 10. Security and Privacy
- 11. Testing and Quality Assurance
- 12. Setup and Deployment Guide
- 13. Limitations and Risks
- 14. Future Enhancements
- 15. Conclusion

1. Executive Summary

The Personal Finance Advisor Bot is designed to help users understand and manage their everyday finances through a single web interface. Users can record income and expenses, categorize transactions, set spending limits, review summaries, and ask a conversational assistant for explanations or practical budgeting ideas.

The application combines conventional financial calculations with AI-generated explanations. The database and backend should remain the source of truth for numerical totals; the language model should explain results and offer suggestions rather than inventing balances or calculating critical figures without validation.

Expected value

- Makes spending patterns easier to understand through categorized records and visual summaries.
- Helps users compare actual spending with a budget they set.
- Encourages realistic saving habits using user-defined goals and what-if scenarios.
- Provides a friendly conversational interface for common personal-finance questions.
- Creates a foundation for later additions such as recurring transactions, reports, and voice interaction.

2. Problem Statement and Objectives

Problem statement

Many people record expenses inconsistently and may not know which categories consume most of their income. Financial information can be scattered across notes, spreadsheets, and banking apps. A simple personal finance tool can centralize manual records and turn them into understandable summaries.

Objectives

- Provide a simple way to add, edit, view, and delete income and expense entries.
- Organize transactions by category, date, amount, and description.
- Calculate totals, remaining budget, and savings using deterministic backend logic.
- Show monthly summaries and clear visual indicators of spending trends.
- Offer context-aware AI explanations based on selected or summarized user data.
- Protect financial records and clearly communicate the limitations of AI-generated guidance.

3. Scope and Key Features

Feature	Description	Priority
Transaction management	Add, edit, view, and delete income or expense records.	Core
Categories	Group records into categories such as food, travel, bills, education, and salary.	Core
Budget planner	Set a monthly or category-specific budget and compare it with recorded spending.	Core
Dashboard	Display income, expenses, net balance, savings estimate, and category breakdown.	Core
AI finance companion	Answer general questions and explain patterns using approved summaries of the user's records.	Core / API dependent
Savings goals	Set a target amount and deadline, then track progress.	Recommended

Feature	Description	Priority
What-if simulator	Estimate how changing a spending category affects a savings target.	Recommended
Monthly report	Generate a concise summary of totals, category patterns, and budget status.	Recommended
Voice input	Allow speech-to-text entry or questions if browser and API support are available.	Optional

Out of scope for the initial version

- Direct bank-account linking, payment processing, or moving money between accounts.
- Guaranteed investment returns, tax advice, credit approval, or regulated financial advice.
- Automatic access to bank data unless a secure, explicitly authorized integration is implemented.

4. Functional and Non-Functional Requirements

Functional requirements

ID	Requirement	Acceptance check
FR-01	The user can create a transaction with type, amount, date, category, and description.	Valid entry is saved and appears in the transaction list.
FR-02	The user can update or remove a transaction.	The stored record and displayed totals update correctly.
FR-03	The system calculates income and expense totals for a selected period.	Totals match a manually verified test dataset.
FR-04	The user can set a budget.	Dashboard compares recorded spending with the saved budget.
FR-05	The dashboard presents understandable summaries.	Values and charts match backend calculations.
FR-06	The AI assistant can explain a finance question using permitted context.	Response is relevant, clearly qualified, and does not claim unsupported facts.
FR-07	The system validates user input.	Invalid amounts, dates, or missing required fields are rejected with clear messages.

Non-functional requirements

- **Usability:** Common actions should be clear, with readable labels, helpful validation, and a responsive layout.
- **Reliability:** Financial totals must be calculated by tested backend functions, not inferred from AI text.
- **Security:** Secrets must be stored outside source code; user data must be protected by appropriate access controls.
- **Maintainability:** Separate routes, data models, calculation logic, and AI integration into clear modules as the project grows.
- **Performance:** Use database queries and indexes appropriate to the expected volume of transactions.
- **Portability:** Support local SQLite development and a configured PostgreSQL deployment where needed.

5. System Architecture

The application follows a layered architecture. The browser sends a request to Flask routes; the backend validates input and invokes service logic; SQLAlchemy reads or writes records; and the AI service is called only when a conversational response is needed.

Presentation layer HTML, CSS, Jinja2 templates, dashboard charts
↓
Application layer Flask routes, validation, session/authentication if configured
↓
Service layer Budget calculations, reporting, savings simulation, AI prompt/context builder

■■	
Data layer SQLAlchemy models and database	**AI integration** Configured language-model API

Recommended request flow

- 1. Receive the request through a Flask route.
- 2. Validate and normalize the input, including amount, date, and category.
- 3. Apply the required business rules and perform calculations in Python.
- 4. Save or retrieve records through SQLAlchemy.
- 5. If AI assistance is requested, send only the minimum relevant context to the configured AI provider.
- 6. Validate the response, present it clearly, and handle provider errors gracefully.

6. Technology Stack

Technology	Role	Notes
Python	Core programming language	Use a supported version and a virtual environment.
Flask	Web server and routing	Organize routes and configuration cleanly.
SQLAlchemy	Object-relational mapping	Defines models and manages database access.
SQLite	Local development database	Simple to start with for a single-user prototype.
PostgreSQL	Deployment database option	Recommended when multi-user access and production operation are required.
HTML / CSS / Jinja2	User interface	Use consistent forms, navigation, and responsive layout.
Chart library	Visual summaries	Optional; chart values should come from backend aggregates.
Gemini or other LLM API	Conversational assistance	Requires an API key, provider error handling, and usage controls.

7. Main User Workflows

A. Record a transaction

- The user opens the transaction form and selects income or expense.
- The user enters amount, date, category, and an optional note.
- The server validates the values and saves the record.
- The transaction list and dashboard refresh using database-backed totals.

B. Review the monthly budget

- The user chooses a month and views recorded income and expenses.
- The backend sums expenses by category and compares them with saved budgets.
- The dashboard shows used budget, remaining budget, and any over-budget categories.
- The user can review the figures and decide whether to adjust the budget.

C. Ask the AI finance companion

- The user asks a question such as, “How can I reduce my food spending?”
- The backend retrieves an appropriate summary, if the user has authorized its use.
- The application sends the question and limited context to the AI provider.
- The assistant returns suggestions, while the UI reminds the user that these are informational and should be checked against their circumstances.

D. Explore a savings scenario

The user selects a category and enters a hypothetical reduction. The backend calculates the estimated monthly and annual difference using the selected amount and period. The application labels the result as an estimate rather than a guarantee.

8. Data Model

A minimal database schema may include the following entities. Actual table names and fields should be adjusted to match the implementation.

Entity	Suggested fields	Purpose
User (if accounts are supported)	id, name, email, password_hash, created_at	Separates records by user and supports authentication.
Transaction	id, user_id, type, amount, category, description, transaction_date, created_at	Stores income and expense entries.
Budget	id, user_id, category, amount_limit, period_month, period_year	Stores a user's budget limits.
SavingsGoal	id, user_id, title, target_amount, current_amount, target_date	Tracks progress toward a savings goal.
ChatLog (optional)	id, user_id, question, response_summary, created_at	Stores limited conversation history if needed and disclosed.

Data integrity guidelines

- Store monetary values as decimal/numeric values rather than binary floating-point where practical.
- Validate that transaction amounts are positive and that the transaction type is allowed.
- Use foreign keys and user-scoped queries if the application supports multiple accounts.
- Keep timestamps consistent and document the timezone used for reporting.
- Do not store raw API keys, passwords, or unnecessary sensitive chat content in the database.

9. AI Assistant Design and Safety

The AI assistant should complement the application's deterministic calculations. It can explain trends, help users think through spending choices, and answer general financial-literacy questions. It should not be treated as the authoritative source for account balances, totals, or financial facts.

Recommended design rules

- Calculate totals, budget status, and projections in tested Python functions before calling the AI.
- Send only the context needed for the current question; avoid exposing full transaction history by default.
- Ask the model to distinguish observed data from assumptions and general suggestions.
- Handle timeouts, rate limits, missing API keys, and malformed provider responses.
- Do not promise savings, returns, debt outcomes, or other results that cannot be guaranteed.
- Show a brief notice that AI responses can be inaccurate and are for educational purposes.

Example interaction

User: “I spent ■3,000 on food this month. How can I reduce it?”
Application: Retrieves the user's food budget and current spending from the database.
Assistant: Explains whether spending is above the saved budget and offers practical options, clearly identifying any assumptions.

10. Security and Privacy

Financial records are sensitive. Even a student prototype should follow basic secure-development practices, especially if it is deployed publicly.

Area	Recommended control
Secrets	Load AI keys and secret values from environment variables; never commit them to GitHub.
Authentication	If accounts are enabled, hash passwords with a suitable password-hashing library and use secure session handling.
Authorization	Verify that each request can access only the current user's records.
Input validation	Validate and constrain form fields on the server; do not rely only on browser validation.
Database	Use ORM parameterization, migrations, backups, and least-privilege database credentials.
Transport	Use HTTPS in deployment and secure cookie settings where sessions are used.
AI privacy	Disclose AI-provider processing and send the minimum necessary financial context.
Logging	Avoid logging passwords, tokens, full financial records, or sensitive prompts.

Privacy note

Before using real personal financial data, review the selected AI provider's data-handling terms, define retention rules, and make it clear to users what is stored and what may be sent to an external service.

11. Testing and Quality Assurance

Testing should cover financial calculations, data validation, database operations, route behavior, and AI-provider failure cases. Use synthetic sample data during development.

Test area	Example test	Expected result
Calculations	Income ■25,000 and expenses ■12,000	Net difference is ■13,000.
Validation	Submit a negative transaction amount	Request is rejected with a clear validation message.
Budget	Budget ■5,000; eligible expenses ■5,500	Spent amount is ■5,500 and over-budget amount is ■500.
Date filtering	Create transactions in two different months	Monthly report includes only the selected period.
CRUD	Edit and then delete a transaction	List and totals reflect the latest database state.
Authorization	Attempt to access another user's record, if accounts exist	Access is denied.
AI unavailable	Simulate a provider timeout	User sees a safe fallback message; app remains usable.
AI correctness	Ask for a total already calculated by backend	Response uses the supplied total or links to displayed data; it does not invent a different total.

Release checklist

- All core calculations have unit tests.
- Forms show validation errors and preserve usability on mobile.
- Database configuration is externalized and tested.
- AI provider errors do not crash the application.
- Secrets are absent from the repository and deployment logs.
- README includes installation steps, configuration, screenshots, limitations, and known issues.
- Deployment is tested with non-sensitive sample data.

12. Setup and Deployment Guide

The following is a general setup workflow for a Flask-based implementation. File names and commands may need to be adjusted to match the actual repository.

Prerequisites

- Python installed and available from the terminal.
- Git installed if cloning or version-controlling the project.
- A configured database; SQLite is suitable for local development.
- An AI-provider API key only if AI features are enabled.

Typical local setup

```
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

Set configuration values through environment variables, initialize the database using the project's migration or initialization process, and start the Flask development server. Do not use Flask's development server as the production server.

Configuration examples

Variable	Purpose
SECRET_KEY	Secret used by Flask sessions or related security features.
DATABASE_URL	Database connection string, if the application uses environment-based configuration.
AI_API_KEY	API key for the configured AI provider; use the provider-specific variable name in the code.
FLASK_ENV / app config	Use the configuration approach supported by the installed Flask version; do not rely on obsolete settings.

Deployment recommendations

- Use a production WSGI server and a hosting platform that supports Python applications.
- Configure PostgreSQL and secure environment variables for production if required.
- Enable HTTPS, database backups, error monitoring, and appropriate access restrictions.
- Keep development and production configuration separate.
- Run tests and verify migrations before releasing an update.

13. Limitations and Risks

Limitation / risk	Mitigation
AI can produce incorrect or overconfident answers.	Use backend-calculated figures, clear disclaimers, constrained prompts, and user review.
Manual entry may be incomplete or inaccurate.	Allow correction, show transaction history, and clearly label summaries as based on recorded data.
A prototype may lack multi-user security.	Implement authentication and authorization before storing different users' records.

Limitation / risk	Mitigation
External AI services may be unavailable or incur usage costs.	Provide a fallback response, set usage limits, and document provider requirements.
Budget calculations are estimates and depend on recorded data.	Explain assumptions and avoid presenting projections as guaranteed outcomes.
Public deployment can expose secrets or private records if misconfigured.	Review configuration, permissions, logs, and access controls before launch.

14. Future Enhancements

- Recurring transactions and reminders for bills or subscriptions.
- Export monthly reports to PDF or CSV.
- More detailed visualizations and comparisons across months.
- Goal-based savings planner with progress tracking.
- Multilingual chat and speech-to-text input, subject to browser and provider support.
- Receipt upload and OCR, with user confirmation before saving extracted amounts.
- Optional read-only bank-data integration through a trusted provider, only with explicit consent.
- A test suite, CI workflow, structured logs, and deployment monitoring.

15. Conclusion

The Personal Finance Advisor Bot can serve as a practical full-stack project demonstrating Python web development, database design, financial calculations, dashboard design, and AI API integration. Its strongest design principle is to keep financial arithmetic deterministic and auditable while using AI for explanation and guidance. Before presenting the project as complete, verify each feature against the actual code, test it with sample data, and document any unfinished functionality.

Implementation status reminder
This PDF documents the planned system and recommended engineering practices. It does not certify that every listed feature is already implemented, tested, or deployed.